ETANA OROKO

Milestone 01 - MVP Delivery

Technical Documentation

UI + Backend Integration + Security

Version 1.0.0

April 2026

Platform: Android & iOS

Table of Contents

1. Project Overview

Etana Oroko is a community-driven social networking application designed to connect members through posts, interactions, and real-time communication in one unified platform.

This document covers Milestone 01 (MVP), which delivers a fully functional authentication flow, a social feed with posts/likes/comments, user profiles, and a secure Firebase backend - all built on Clean Architecture principles.

App Name	Etana Oroko
Version	1.0.0+1
Platforms	Android & iOS
Milestone	01 - MVP Delivery

---

2. Technology Stack

Framework & Language

- Flutter (Dart) - cross-platform mobile development
- Dart SDK ^3.11.0 with strict null safety

Backend & Cloud

- Firebase Authentication (Email/Password + Google Sign-In)
- Cloud Firestore (NoSQL document database)
- Firebase Core ^4.4.0

Architecture & State

- Clean Architecture (Feature-based, 3-layer)
- Provider (^6.1.5) - State Management
- GetIt (^9.2.1) - Dependency Injection
- GoRouter (^17.1.0) - Declarative Navigation

Key Packages

- google_sign_in ^7.2.0 - Google OAuth
- connectivity_plus ^7.0.0 - Network monitoring
- cached_network_image ^3.4.1 - Image caching
- flutter_secure_storage ^10.0.0 - Secure local storage
- flutter_svg ^2.2.3 - SVG asset rendering

Design System

- Poppins font family (Regular 400, Medium 500, SemiBold 600)
- Custom color palette (Brand Blue #2563EB primary)
- Responsive design with mobile/tablet breakpoints

3. Architecture & Project Structure

The application follows Clean Architecture with a strict 3-layer dependency rule: Presentation --> Domain <-- Data. The Domain layer is pure Dart with no Flutter or Firebase imports.

Dependency Flow

UI (Widgets) --> Provider --> Use Case --> Repository Interface --> Data Source --> Firebase

Project Directory Layout

```
lib/
 +-- main.dart
 +-- app/
 |   +-- app_bootstraps.dart
 |   +-- etanaoroko_app.dart
 |   +-- injection_container.dart
 +-- core/
 |   +-- config/         (responsive config)
 |   +-- constants/      (app assets, constants)
 |   +-- enums/          (app-wide enums)
 |   +-- errors/         (failure sealed classes)
 |   +-- extensions/     (context, string, widget, responsive)
 |   +-- providers/      (theme provider)
 |   +-- router/         (GoRouter, route names, transitions)
 |   +-- services/       (firebase, logger, network, storage, etc.)
 |   +-- theme/          (colors, fonts, text styles, shadows, borders)
 |   +-- utils/          (system utils, validators)
 |   +-- widgets/        (animated, buttons, cards, dialogs, inputs, etc.)
 +-- features/
     +-- auth/
     |   +-- data/       (models, datasources, repositories)
     |   +-- domain/     (entities, repositories, usecases)
     |   +-- presentation/ (providers, screens, widgets)
     +-- feed/
     |   +-- data/       (models, datasources, repositories)
     |   +-- domain/     (entities, repositories, usecases)
     |   +-- presentation/ (providers, screens, widgets)
     +-- profile/
     |   +-- data/       (models, datasources, repositories)
     |   +-- domain/     (entities, repositories, usecases)
     |   +-- presentation/ (providers, screens, widgets)
     +-- splash/
         +-- presentation/ (screens, widgets)
```

4. UI Screens Developed

All screens were designed and built with custom widgets, animations, and the Poppins-based design system. Below is a summary of each screen delivered in the MVP.

4.1 Splash Screen

- Animated brand logo with loading indicator
- Auth-aware navigation: routes to Feed (if logged in) or Login (if not)
- 3-second display duration with fade transition

4.2 Login Screen

- Email and password input fields with real-time validation
- "Sign In" button with loading state
- "Continue with Google" button for Google OAuth sign-in
- "Create Account" navigation link
- Error display via snackbar with user-friendly messages

4.3 Create Account Screen

- Name, email, password, and confirm password fields
- Comprehensive form validation (email format, password strength, match check)
- "Create Account" button with loading state
- Success auto-redirects to Feed screen

4.4 Feed Screen

- Post list with pull-to-refresh and loading states
- Create post section with text input and post button
- Each post card shows: author avatar (photo or initials), name, timestamp, content
- Like toggle with optimistic UI update and count display
- Expandable comments section with reply input
- Empty state and error state handling
- App bar with profile navigation button

4.5 Profile Screen

- User avatar (Google photo or initial-based)
- Display name and email from Firestore
- Activity stats: total posts, total likes received, total comments
- User's posts list with like/comment actions
- Supports viewing own profile or another user's profile

5. Authentication - Backend Integration

Full Firebase Authentication integration with email/password and Google Sign-In support. The auth flow creates a Firestore user document on signup and validates credentials on login.

Domain Layer

- UserEntity - uid, name, email, photoUrl, createdAt, updatedAt
- AuthRepository (abstract interface) - single contract for all auth operations
- 5 Use Cases: SignIn, SignInWithGoogle, CreateAccount, SignOut, GetAuthState

Data Layer

- UserModel - extends UserEntity with Firestore serialization (fromFirestore, toFirestore, fromFirebaseUser)
- AuthRemoteDataSource - wraps Firebase Auth + Firestore calls for user management
- AuthRepositoryImpl - implements AuthRepository with network checks and error mapping

Presentation Layer

- LoginProvider - manages sign-in state (loading, success, error) via use cases
- CreateAccountProvider - manages account creation state via use cases
- Both providers are registered as factories in DI

Sign-In Flow

1. User enters email + password --> form validation
2. LoginProvider calls SignInUseCase
3. Repository checks network --> calls Firebase Auth
4. On success: fetches user doc from Firestore --> returns UserEntity
5. Provider sets isSuccess --> UI navigates to /feed
6. On failure: AuthFailure.fromCode() maps error --> snackbar message

Google Sign-In Flow

1. User taps "Continue with Google"
2. Native Google Sign-In dialog opens
3. On consent: GoogleAuthCredential --> Firebase signInWithCredential
4. Check if user doc exists in Firestore
5. New user --> create doc with Google profile (name, email, photoUrl)
6. Existing user --> fetch doc
7. Provider sets isSuccess --> UI navigates to /feed

Create Account Flow

1. User enters name + email + password + confirm password
2. Form validation: name >=2 chars, valid email, password strength, match
3. CreateAccountProvider calls CreateAccountUseCase
4. Firebase Auth creates user --> updates displayName
5. Creates Firestore doc: users/{uid} with serverTimestamp
6. Provider sets isSuccess --> UI navigates to /feed

6. Social Feed - Backend Integration

Complete Firestore-backed social feed with paginated posts, likes (sub-collection), and comments (sub-collection). Optimistic UI updates for likes ensure a responsive experience.

Domain Layer

- PostEntity - id, userId, userName, userPhotoUrl, content, createdAt, likesCount, commentsCount, isLiked
- CommentEntity - id, postId, userId, userName, userPhotoUrl, content, createdAt
- FeedRepository interface - getPosts, createPost, toggleLike, getComments, addComment, getUserPosts
- 6 Use Cases: GetPosts, CreatePost, ToggleLike, GetComments, AddComment, GetUserPosts

Data Layer

- PostModel - extends PostEntity with Firestore serialization
- CommentModel - extends CommentEntity with Firestore serialization
- FeedRemoteDataSource - handles all Firestore queries and writes for feed
- FeedRepositoryImpl - implements FeedRepository with network checks, like-status resolution, error mapping

Key Operations

- Get Posts: Firestore query ordered by createdAt DESC, limit 20, with per-post like-status check
- Create Post: Auto-ID document with server timestamp, denormalized user data
- Toggle Like: Batch write - set/delete likes/{uid} sub-doc + increment/decrement likesCount
- Get Comments: Query posts/{postId}/comments ordered by createdAt ASC
- Add Comment: Batch write - add comment doc + increment commentsCount

Feed Provider State Management

```
FeedProvider manages:
- List<PostEntity> _posts - cached feed data
- bool _isLoading - loading indicator state
- bool _isCreatingPost - post creation loading state
- String? _errorMessage - latest error message

Optimistic Like Toggle:
1. Immediately toggle isLiked + adjust likesCount in local list
2. Call ToggleLikeUseCase in background
3. On failure: revert local state + show error snackbar
```

7. User Profile - Backend Integration

Profile screen fetches user data from Firestore and computes activity statistics by aggregating the user's posts.

Domain Layer

- ProfileEntity - uid, name, email, photoUrl, bio, createdAt, updatedAt
- ProfileRepository interface - getUserProfile, getCurrentUserProfile, getUserStats
- ProfileStats - totalPosts, totalLikes, totalComments
- 3 Use Cases: GetProfile, GetCurrentProfile, GetUserStats

Data Layer

- ProfileModel - extends ProfileEntity with Firestore serialization
- ProfileRemoteDataSource - fetches user documents and computes stats
- ProfileRepositoryImpl - implements ProfileRepository with network checks and error mapping

Profile Provider

- Loads profile, stats, and user posts in parallel
- Supports viewing own profile (no userId) or another user's profile (with userId)
- Manages loading, data, and error states

---

8. Firestore Data Schema

Collection: users/{uid}

Field	Type	Required	Description
uid	string	Yes	Same as document ID
name	string	Yes	Display name
email	string	Yes	Email address
photoUrl	string	No	Google profile photo URL
createdAt	Timestamp	Yes	Server timestamp
updatedAt	Timestamp	No	Server timestamp on update

Collection: posts/{postId}

Field	Type	Required	Description
id	string	Yes	Same as document ID
userId	string	Yes	Author UID
userName	string	Yes	Author display name
userPhotoUrl	string	No	Author photo URL
content	string	Yes	Post text content

9. Use Cases Summary

Each use case encapsulates a single business operation. They validate inputs, delegate to the repository interface, and are the only entry point providers use to trigger backend operations.

Authentication Use Cases

Use Case	Input	Output
SignInUseCase	email, password	UserEntity
SignInWithGoogleUseCase	(none)	UserEntity
CreateAccountUseCase	name, email, password	UserEntity
SignOutUseCase	(none)	void
GetAuthStateUseCase	(none)	Stream<UserEntity?>

Feed Use Cases

Use Case	Input	Output
GetPostsUseCase	limit, lastPost?	List<PostEntity>
CreatePostUseCase	content	PostEntity
ToggleLikeUseCase	postId, isCurrentlyLiked	PostEntity
GetCommentsUseCase	postId	List<CommentEntity>
AddCommentUseCase	postId, content	CommentEntity
GetUserPostsUseCase	userId	List<PostEntity>

Profile Use Cases

Use Case	Input	Output
GetProfileUseCase	uid	ProfileEntity
GetCurrentProfileUseCase	(none)	ProfileEntity
GetUserStatsUseCase	uid	ProfileStats

10. Dependency Injection

All dependencies are registered centrally via GetIt in injection_container.dart. Each feature has its own DI module (auth_di.dart, feed_di.dart, profile_di.dart).

Registration Strategy

- Core Firebase services - registerLazySingleton (single instance)
- Data sources - registerLazySingleton
- Repositories (as abstract interfaces) - registerLazySingleton
- Use cases - registerLazySingleton

11. Error Handling & Failure Mapping

A sealed Failure class hierarchy provides typed error handling across the entire application. Raw Firebase exceptions are never exposed to end users.

Failure Types

Failure Class	Source	Example
AuthFailure	Firebase Auth	wrong-password, email-already-in-use
ServerFailure	Cloud Firestore	permission-denied, not-found
NetworkFailure	Connectivity	No internet connection
ValidationFailure	Use Case input	Empty post content

Auth Error Mappings

Firebase Code	User-Friendly Message
wrong-password	Incorrect password. Please try again.
user-not-found	No account found with this email.
email-already-in-use	An account with this email already exists.
invalid-email	Please enter a valid email address.
too-many-requests	Too many attempts. Please try again later.
network-request-failed	No internet connection. Please check your network.
weak-password	Password is too weak. Please choose a stronger password.
sign_in_canceled	Sign-in was cancelled.
(default)	Something went wrong. Please try again.

Error Flow

```
Firebase SDK throws exception
  --> Data source catches --> rethrows raw exception
  --> Repository catches --> maps to Failure subclass with user-friendly message
  --> Use case propagates Failure
  --> Provider catches Failure --> sets errorMessage
  --> UI shows snackbar with errorMessage
```

12. Firestore Security Rules

All Firestore security rules are versioned in the repository (firestore.rules). The rules enforce authentication, ownership, and field validation at the database level.

Users Collection - /users/{uid}

- Read: Any authenticated user can read any user profile
- Create: Only the owner (auth.uid == uid) with required fields validated (uid, name, email, createdAt)
- Update: Only the owner (auth.uid == uid)
- Delete: Denied in MVP

Posts Collection - /posts/{postId}

- Read: Any authenticated user
- Create: Authenticated users, only for themselves (userId == auth.uid), required fields validated
- Update: Post author can update anything; other authenticated users can only update likesCount/commentsCount
- Delete: Only the post author

Likes Sub-collection - /posts/{postId}/likes/{likeId}

- Read: Any authenticated user
- Create: Only if likeId == auth.uid (user can only create their own like)
- Delete: Only if likeId == auth.uid (user can only remove their own like)
- Update: Denied

Comments Sub-collection - /posts/{postId}/comments/{commentId}

- Read: Any authenticated user
- Create: Authenticated users, only for themselves (userId == auth.uid), required fields validated
- Update: Only the comment author
- Delete: Only the comment author

Default Rule

```
match /{document=**} {
  allow read, write: if false;
}
```

All paths not explicitly matched are denied by default.

Security Principles

- No unauthenticated access to any data
- Users cannot impersonate other users in writes
- Required fields are validated at the database level
- No broad permissive catch-all rules
- Rules are version-controlled and reviewed with schema changes

13. App Navigation & Auth Routing

GoRouter manages all navigation with auth-aware redirects. The splash screen checks Firebase Auth state and routes accordingly.

Routes

Path	Screen	Auth Required
/splash	Splash Screen	No (checks auth)
/login	Login Screen	No
/create-account	Create Account Screen	No
/feed	Feed Screen	Yes
/profile	Profile Screen	Yes

Auth Redirect Logic

- Splash: no redirect - handles its own auth-aware navigation
- Not logged in + protected route --> redirected to /login
- Logged in + auth page (login/create-account) --> redirected to /feed

---

14. Core Services & Utilities

- FirebaseAuthService - wraps all FirebaseAuth SDK calls; single import point for auth
- FirestoreService - wraps all Firestore SDK calls; provides typed collection references
- LoggerService - centralized logging with class context (info, warning, error, success)
- NetworkService - connectivity monitoring via connectivity_plus
- LocalStorageService - flutter_secure_storage wrapper for small session flags
- Validators - email, password strength, name, and content validation utilities
- SystemUtils - system UI configuration and orientation lock
- ResponsiveConfig - mobile/tablet breakpoint management

---

15. What Is NOT Included (Post-MVP)

The following features are intentionally excluded from Milestone 01 and planned for future milestones:

- Direct messaging / chat system
- Push notifications (Firebase Messaging is imported but not wired)
- Full comments and likes advanced features (threading, reactions)
- Advanced profile editing (bio, photo upload)
- Media upload pipeline (images/videos in posts)
- Friend/follow graph
- Search functionality
- Dark mode (color stubs exist but not implemented)
- Firebase Crashlytics / Analytics integration